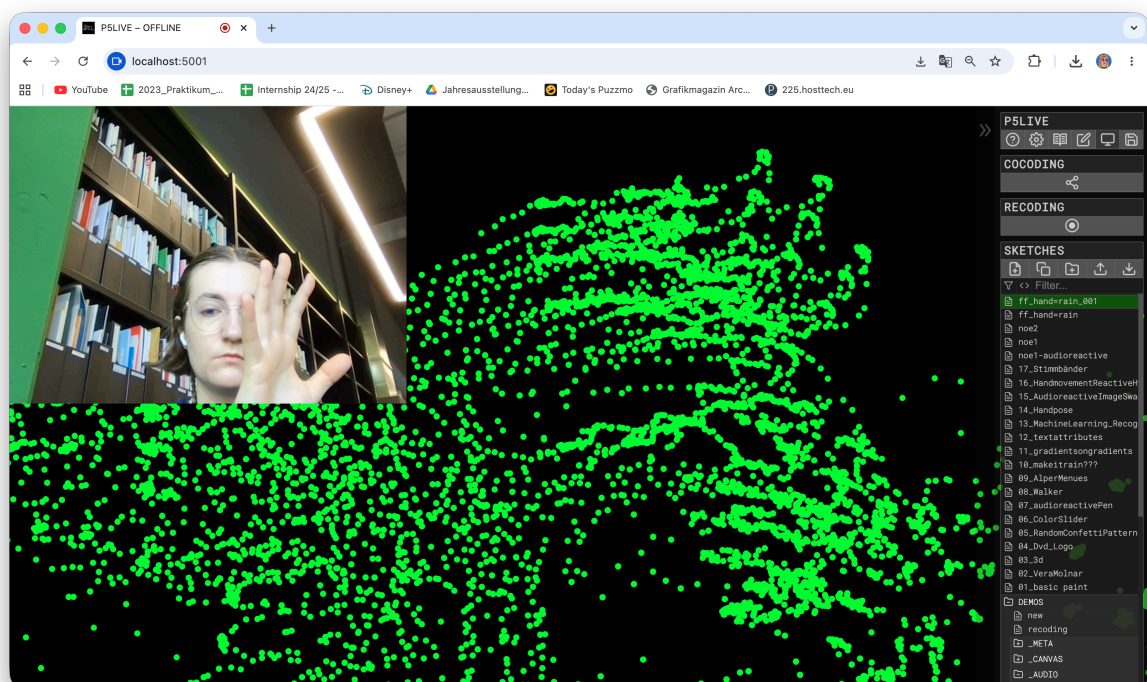


Handtracking



```

let libs = [
  'https://unpkg.com/ml5@1/dist/ml5.min.js',
  'https://unpkg.com/hydra-synth',
  'https://cdn.jsdelivr.net/gh/fffd8/hy5@main/hy5.js'
]

let handPose;
let video;
let hands = [];

function preload() {
  // Load the handPose model
  handPose = ml5.handPose();
}

function setup() {
  createCanvas(windowWidth, windowHeight);
  video = createCapture(VIDEO, { flipped: true });
  // Create the webcam video and hide it
  video = createCapture(VIDEO);
  video.size(windowWidth, windowHeight);
  video.hide();
  // start detecting hands from the webcam video
  handPose.detectStart(video, gotHands);
}

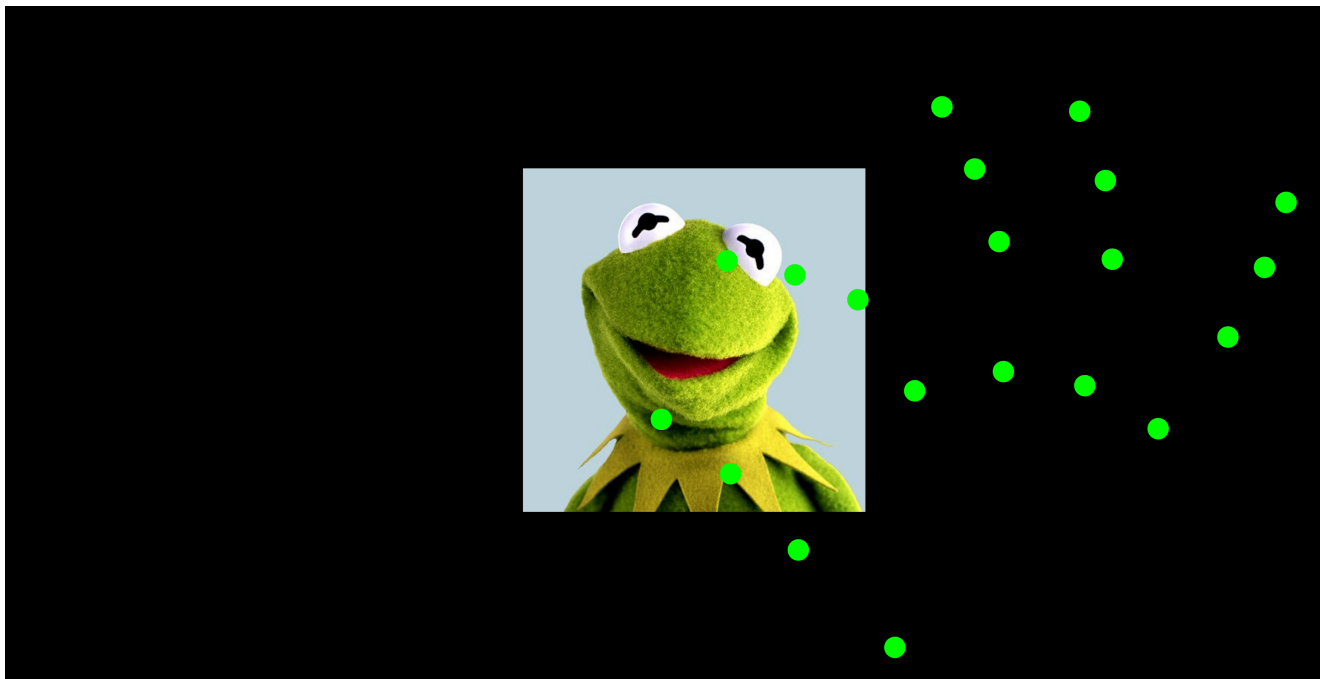
function draw() {
  // Draw the webcam video
  //image(video, 0, 0, width, height);

  // Draw all the tracked hand points
  for (let i = 0; i < hands.length; i++) {
    let hand = hands[i];
    for (let j = 0; j < hand.keypoints.length; j++) {
      let keypoint = hand.keypoints[j];
      fill(0, 255, 0);
      noStroke();
      circle(keypoint.x, keypoint.y, 10);
    }
  }
}

// Callback function for when handPose outputs data
function gotHands(results) {
  // save the output to the hands variable
  hands = results;
}

```

Handpuppet



```
let libs = [  
  'https://unpkg.com/ml5@1/dist/ml5.min.js',  
  'https://unpkg.com/hydra-synth',  
  'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js'  
]  
  
let handPose;  
let video;  
let hands = [];  
let kermit;  
  
function preload() {  
  handPose = ml5.handPose();  
  kermit = loadImage(  
    'https://lumiere-a.akamaihd.net/v1/images/character_themuppets_kermit_b77a431b.jpeg');  
}  
  
function setup() {  
  createCanvas(windowWidth, windowHeight);  
  
  video = createCapture(VIDEO);  
  video.size(windowWidth, windowHeight);  
  video.hide();  
  
  handPose.detectStart(video, gotHands);  
}  
  
function draw() {  
  background(0);
```

```

translate(width, 0);
scale(-1, 1);

for (let i = 0; i < hands.length; i++) {

  let hand = hands[i];
  // fingertip points
  let thumb = hand.thumb_tip;
  let index = hand.index_finger_tip;
  // midpoint between fingers
  let x = (thumb.x + index.x) / 2;
  let y = (thumb.y + index.y) / 2;
  // distance between fingers
  let d = dist(thumb.x, thumb.y, index.x, index.y);
  // image size reacts to pinch distance
  let size = d * 2;

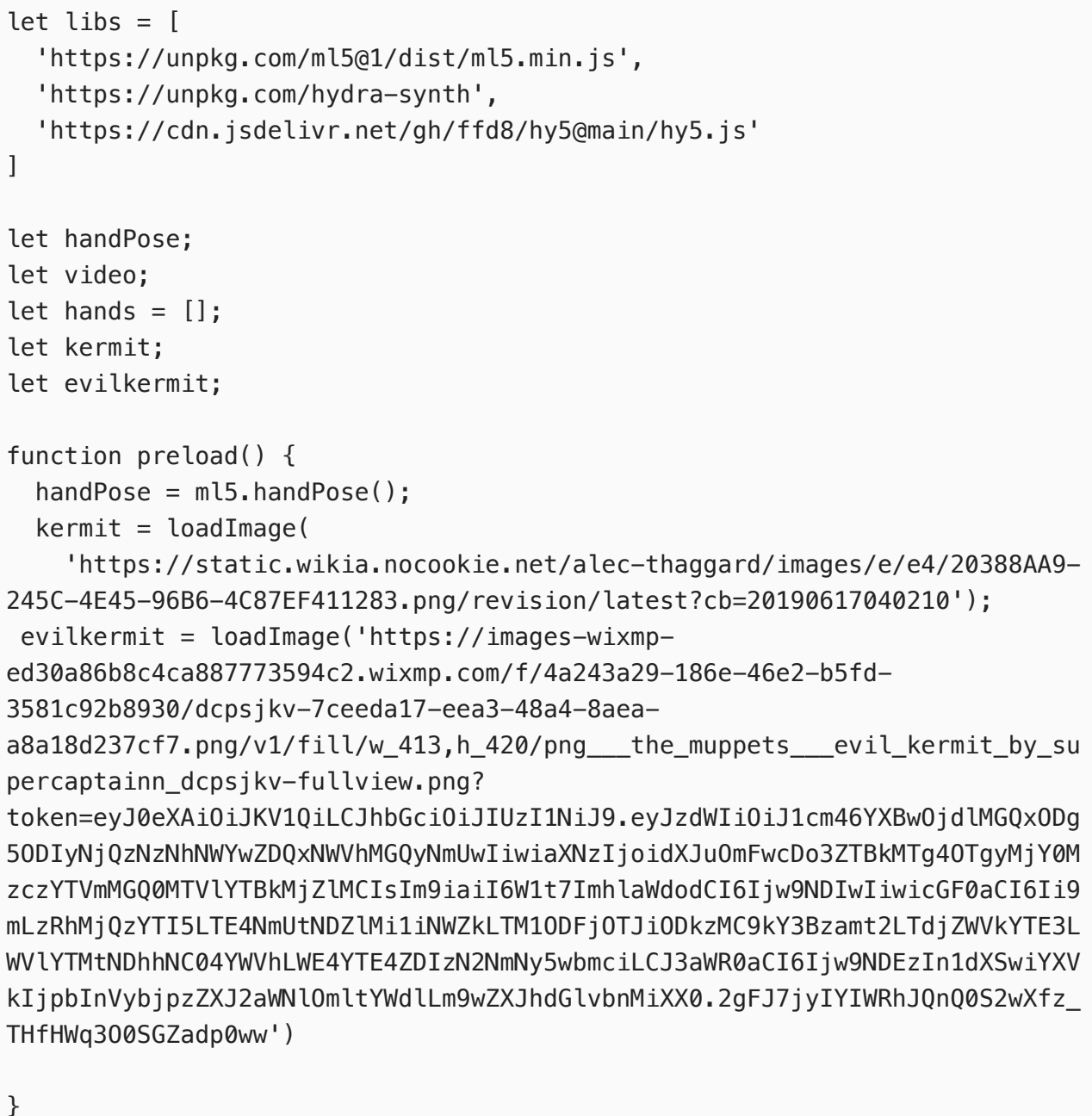
  imageMode(CENTER);
  image(kermit, x, y, size, size);
}
// draw hand points
for (let i = 0; i < hands.length; i++) {
  let hand = hands[i];

  for (let j = 0; j < hand.keypoints.length; j++) {
    let keypoint = hand.keypoints[j];

    fill(0, 255, 0);
    noStroke();
    circle(keypoint.x, keypoint.y, 30);
  }
}

function gotHands(results) {
  hands = results;
}

```



```

function setup() {
  createCanvas(windowWidth, windowHeight);

  video = createCapture(VIDEO);
  video.size(windowWidth, windowHeight);
  video.hide();

  handPose.detectStart(video, gotHands);
}

function draw() {
  background(0);

  translate(width, 0);
  scale(-1, 1);

  for (let i = 0; i < hands.length; i++) {

    let hand = hands[i];

    let thumb = hand.keypoints[4];
    let index = hand.keypoints[8];

    let x = (thumb.x + index.x) / 2;
    let y = (thumb.y + index.y) / 2;

    let d = dist(thumb.x, thumb.y, index.x, index.y);
    let size = d * 2;

    imageMode(CENTER);

    if (hand.handedness === "Left") {
      image(kermit, x, y, size, size);
    } else if (hand.handedness === "Right") {
      image(evilkermit, x, y, size, size);
    }
  }

  // draw hand points
  for (let i = 0; i < hands.length; i++) {
    let hand = hands[i];

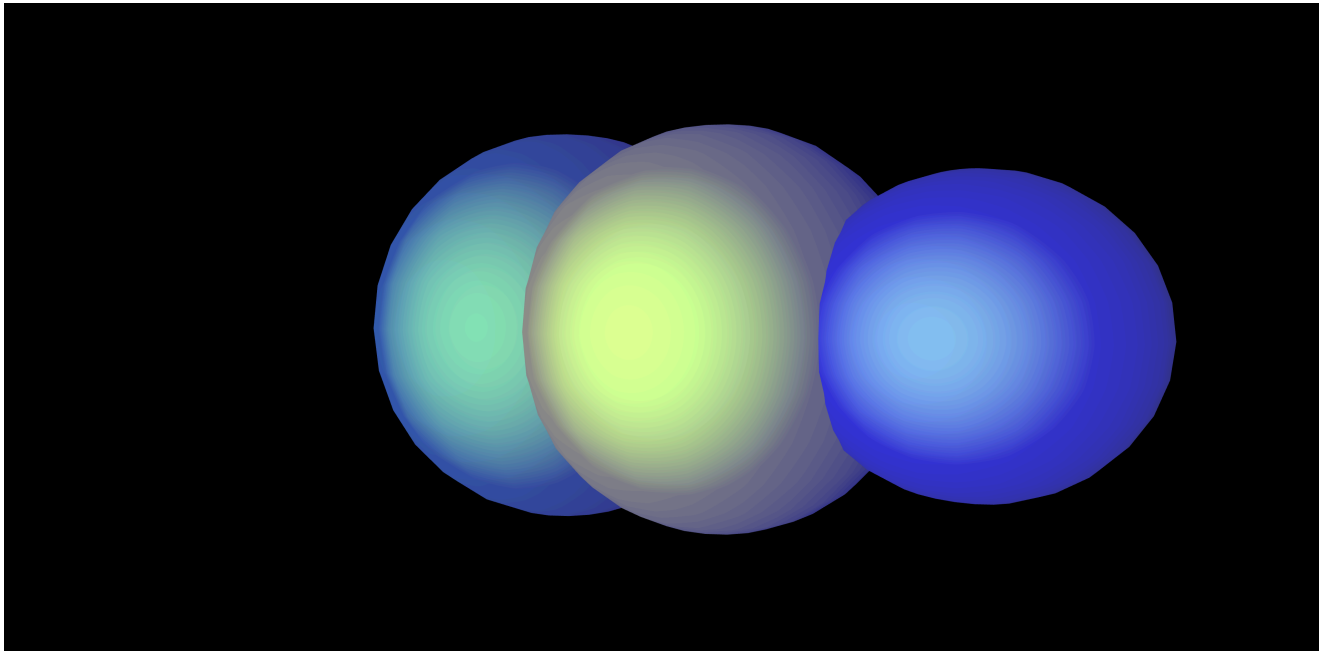
    for (let j = 0; j < hand.keypoints.length; j++) {
      let keypoint = hand.keypoints[j];

      fill(0, 255, 0);
      noStroke();
      circle(keypoint.x, keypoint.y, 30);
    }
  }
}

```

```
}  
  
function gotHands(results) {  
  hands = results;  
}
```

3D, Lights (with Yann)



```
function setup() {  
  createCanvas(windowWidth, windowHeight, WEBGL)  
  
  // audio stuff now behind the scenes, 'true' makes class vars global  
  setupAudio(true) // if empty, use 'a5.' before audio vars below  
  // a5.ease = .075 // customize ease speed  
}  
  
function draw() {  
  /* audio vars: amp, ampL, ampR, ampEase, fft, fftEase, waveform,  
  waveformEase */  
  updateAudio()  
  orbitControl()  
  lights()  
  ambientLight(10);  
  ambientMaterial(100,100,255);  
  noStroke()  
  
  background(0)  
  fill(0,0,255)  
  
  push()  
  specularMaterial(80,139,11)
```

```

    translate(200, 0)
    sphere(50 + fftEase[100])
    pop()

    push()
    fill(0, 100, 100)
    specularMaterial(80,139,11)
    translate(-100, 0)
    sphere(50 + fftEase[20])
    pop()

    specularMaterial(80,139,11)
    sph(0,fftEase[0] * .7, -50, 50 + fftEase[60])

    /* fftEase */
    for(let i = 0; i < fftEase.length; i++) {
        let freq = fftEase[i]; // (0, 255)
        let x = map(i, 0, fftEase.length, 0, width)
        let w = width / fftEase.length
        rect(x, height * .805, w, freq)
    }
}

function sph(x,y,z, size, rspeed){
    push()
    //rotate(frameCount * .1)
    noStroke()
    fill(255, 255, 0)
    translate(50, 0)
    sphere(50 + fftEase[60])
    pop()
}

/*
P5LIVE - Audio
If using outside P5LIVE, include p5live-audio.js
https://cdn.jsdelivr.net/gh/ffd8/P5LIVE/includes/utils/p5live-audio.js
*/

```